

Shortest Job First (SJF) Non-Preemptive Scheduling

1. Problem Statement

In an operating system emulator, design a non-preemptive Shortest Job First (SJF) CPU scheduler. Given N processes with their respective Arrival Times (AT_i) and Burst Times (BT_i), determine the Completion Time (CT_i), Turnaround Time (TAT_i), and Waiting Time (WT_i) for each process. If a tie occurs in burst times while multiple processes are ready, resolve the tie by picking the process with the earliest arrival time. If arrival times are also identical, break the tie using the lower Process ID (PID).

2. Mathematical Framework

The scheduler evaluates tracking performance metrics using the following deterministic operating system equations:

- **Turnaround Time (TAT_i):** The total elapsed time from process arrival to its complete execution.
$$TAT_i = CT_i - AT_i$$
- **Waiting Time (WT_i):** The cumulative total time a process spends sitting inside the ready queue before processing begins.
$$WT_i = TAT_i - BT_i$$

3. Test Case Specification

Process ID (PID)	Arrival Time (AT_i)	Burst Time (BT_i)
1	0	7
2	2	4
3	4	1
4	5	4

4. Expected Solution Metrics

PID	Arrival Time (\$AT\$)	Burst Time (\$BT\$)	Completion Time (\$CT\$)	Turnaround Time (\$TAT\$)	Waiting Time (\$WT\$)
1	0	7	7	7	0
3	4	1	8	4	3
2	2	4	12	10	6
4	5	4	16	11	7

5. Architectural C++ Solution

```
#include <iostream>
#include <vector>
#include <algorithm>

struct Process {
    int pid;
    int at; // Arrival Time
    int bt; // Burst Time
    int ct; // Completion Time
    int tat; // Turnaround Time
    int wt; // Waiting Time
    bool is_completed = false;
};

void calculateSJF(std::vector<Process>& processes) {
    int current_time = 0;
    int completed = 0;
    int n = processes.size();

    while (completed != n) {
        int idx = -1;
        int min_bt = 1e9;

        // Find the process with the minimum burst time among ready processes
        for (int i = 0; i < n; ++i) {
            if (processes[i].at <= current_time && !processes[i].is_completed) {
                if (processes[i].bt < min_bt) {
                    min_bt = processes[i].bt;
                    idx = i;
                } else if (processes[i].bt == min_bt) {
                    // Tie-breaker 1: Earliest Arrival Time
                }
            }
        }

        // Execute the selected process
        if (idx != -1) {
            processes[idx].ct = current_time + processes[idx].bt;
            processes[idx].tat = processes[idx].ct - processes[idx].at;
            processes[idx].wt = processes[idx].tat - processes[idx].bt;
            processes[idx].is_completed = true;
            completed++;
            current_time = processes[idx].ct;
        }
    }
}
```

```

        if (processes[i].at < processes[idx].at) {
            idx = i;
        } else if (processes[i].at == processes[idx].at) {
            // Tie-breaker 2: Lower Process ID
            if (processes[i].pid < processes[idx].pid) {
                idx = i;
            }
        }
    }
}

if (idx != -1) {
    processes[idx].ct = current_time + processes[idx].bt;
    processes[idx].tat = processes[idx].ct - processes[idx].at;
    processes[idx].wt = processes[idx].tat - processes[idx].bt;
    processes[idx].is_completed = true;

    current_time = processes[idx].ct;
    completed++;
} else {
    // If no process has arrived, advance the system clock
    current_time++;
}
}
}

```

6. Implementation Explanation

The solution relies on an allocation selection loop that simulates a monotonically increasing system clock counter (`current\_time`). During each iteration, the algorithm performs an $O(N)$ inspection across the data array to identify valid candidate processes that have already arrived at the current timestamp context. By managing the ready state using an explicitly checked `is\_completed` boolean flag, the scheduling engine ensures that processes do not suffer from re-execution loops. This design guarantees predictable behavior and prevents deadlocks during process evaluation phases.